Topic Modeling with NMF and SVD

The problem

Topic modeling is a fun way to start our study of NLP. We will use two popular **matrix decomposition techniques**.

We start with a **term-document matrix**:

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth . . .
ANTHONY	157	73	0	0	0	1
BRUTUS	4	157	0	2	0	0
CAESAR	232	227	0	2	1	0
CALPURNIA	0	10	0	0	0	0
CLEOPATRA	57	0	0	0	0	0
MERCY	2	0	3	8	5	8
WORSER	2	0	1	1	1	5

source: Introduction to Information Retrieval

We can decompose this into one tall thin matrix times one wide short matrix (possibly with a diagonal matrix in between).

Notice that this representation does not take into account word order or sentence structure. It's an example of a **bag of words** approach.

Latent Semantic Analysis (LSA) uses Singular Value Decomposition (SVD).

Motivation

Consider the most extreme case - reconstructing the matrix using an outer product of two vectors. Clearly, in most cases we won't be able to reconstruct the matrix exactly. But if we had one vector with the relative frequency of each vocabulary word out of the total word count, and one with the average number of words per document, then that outer product would be as close as we can get.

Now consider increasing that matrices to two columns and two rows. The optimal decomposition would now be to cluster the documents into two groups, each of which has as different a distribution of words as possible to each other, but as similar as possible amongst the documents in the cluster. We will call those two groups "topics". And we would cluster the words into two groups, based on those which most frequently appear in each of the topics.

Getting started

We'll take a dataset of documents in several different categories, and find topics (consisting of groups of words) for them. Knowing the actual categories helps us evaluate if the topics we find make sense.

evaluate if the topics we find make sense.

We will try this with two different matrix factorizations: **Singular Value Decomposition (SVD)** and **Non-negative Matrix Factorization (NMF)**

In [138…

```
import numpy as np
from sklearn.datasets import fetch_20newsgroups
from sklearn import decomposition
from scipy import linalg
import matplotlib.pyplot as plt
```

In [139…

```
%matplotlib inline
np.set_printoptions(suppress=True)
```

Additional Resources

- Data source: Newsgroups are discussion groups on Usenet, which was popular in the 80s and 90s before the web really took off. This dataset includes 18,000 newsgroups posts with 20 topics.
- Chris Manning's book chapter on matrix factorization and LSI
- Scikit learn truncated SVD LSI details

Other Tutorials

- Scikit-Learn: Out-of-core classification of text documents: uses Reuters-21578 dataset (Reuters articles labeled with ~100 categories), HashingVectorizer
- Text Analysis with Topic Models for the Humanities and Social Sciences: uses British and French Literature dataset of Jane Austen, Charlotte Bronte, Victor Hugo, and more

Look at our data

Scikit Learn comes with a number of built-in datasets, as well as loading utilities to load several standard external datasets. This is a great resource, and the datasets include Boston housing prices, face images, patches of forest, diabetes, breast cancer, and more. We will be using the newsgroups dataset.

Newsgroups are discussion groups on Usenet, which was popular in the 80s and 90s before the web really took off. This dataset includes 18,000 newsgroups posts with 20 topics.

In [140…

```
categories = ['alt.atheism', 'talk.religion.misc', 'comp.graphics', 'sci.spa
remove = ('headers', 'footers', 'quotes')
newsgroups_train = fetch_20newsgroups(subset='train', categories=categories,
newsgroups_test = fetch_20newsgroups(subset='test', categories=categories, r
```

In [141…

```
newsgroups_train.filenames.shape, newsgroups_train.target.shape
```

Out[141… ((2034,), (2034,))

Let's look at some of the data. Can you guess which category these messages are in?

In [26]:
```
print("\n".join(newsgroups_train.data[:3]))
```

```
Hi,

I've noticed that if you only save a model (with all your mapping planes
positioned carefully) to a .3DS file that when you reload it after restarting
3DS, they are given a default position and orientation.  But if you save
to a .PRJ file their positions/orientation are preserved.  Does anyone
know why this information is not stored in the .3DS file?  Nothing is
explicitly said in the manual about saving texture rules in the .PRJ file.
I'd like to be able to read the texture rule information, does anyone have
the format for the .PRJ file?

Is the .CEL file format available from somewhere?

Rych


Seems to be, barring evidence to the contrary, that Koresh was simply
another deranged fanatic who thought it neccessary to take a whole bunch of
folks with him, children and all, to satisfy his delusional mania. Jim
Jones, circa 1993.


Nope - fruitcakes like Koresh have been demonstrating such evil corruption
for centuries.

 >In article <1993Apr19.020359.26996@sq.sq.com>, msb@sq.sq.com (Mark Brader)

MB>                                                             So the
MB> 1970 figure seems unlikely to actually be anything but a perijove.

JG>Sorry, _perijoves_...I'm not used to talking this language.

Couldn't we just say periapsis or apoapsis?


hint: definition of perijove is the point in the orbit of a satellite of Jupiter nearest the
planet's center
```

In [27]:
```
np.array(newsgroups_train.target_names)[newsgroups_train.target[:3]]
```

Out[27]: array(['comp.graphics', 'talk.religion.misc', 'sci.space'], dtype='<U18')

The target attribute is the integer index of the category.

In [28]:
```
newsgroups_train.target[:10]
```

Out[28]: array([1, 3, 2, 0, 2, 0, 2, 1, 2, 1])

In [29]:
```
num_topics, num_top_words = 6, 8
```

Stop words, stemming, lemmatization

Stop words

From Intro to Information Retrieval:

Some extremely common words which would appear to be of little value in helping select documents matching a user need are excluded from the vocabulary entirely. These words are called stop words.

The general trend in IR systems over time has been from standard use of quite large stop lists (200-300 terms) to very small stop lists (7-12 terms) to no stop list whatsoever. Web search engines generally do not use stop lists.

NLTK

In [2]:
```
from sklearn.feature_extraction import stop_words

sorted(list(stop_words.ENGLISH_STOP_WORDS))[:20]
```

Out[2]:
```
['a',
 'about',
 'above',
 'across',
 'after',
 'afterwards',
 'again',
 'against',
 'all',
 'almost',
 'alone',
 'along',
 'already',
 'also',
 'although',
 'always',
 'am',
 'among',
 'amongst',
 'amoungst']
```

There is no single universal list of stop words.

Stemming and Lemmatization

from Information Retrieval textbook:

Are the below words the same?

organize, organizes, and organizing

democracy, democratic, and democratization

Stemming and Lemmatization both generate the root form of the words.

Lemmatization uses the rules about a language. The resulting tokens are all actual

words

"Stemming is the poor-man's lemmatization." (Noah Smith, 2011) Stemming is a crude heuristic that chops the ends off of words. The resulting tokens may not be actual words. Stemming is faster.

In [142…
```
import nltk
nltk.download('wordnet')
```

```
[nltk_data] Downloading package wordnet to
[nltk_data]     /home/rachelthomas/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

Out[142… True

In [143…
```
from nltk import stem
```

In [144…
```
wnl = stem.WordNetLemmatizer()
porter = stem.porter.PorterStemmer()
```

In [145…
```
word_list = ['feet', 'foot', 'foots', 'footing']
```

In [146…
```
[wnl.lemmatize(word) for word in word_list]
```

Out[146… ['foot', 'foot', 'foot', 'footing']

In [147…
```
[porter.stem(word) for word in word_list]
```

Out[147… ['feet', 'foot', 'foot', 'foot']

Your turn! Now, try lemmatizing and stemming the following collections of words:

- fly, flies, flying
- organize, organizes, organizing
- universe, university

fastai/course-nlp

Stemming and lemmatization are language dependent. Languages with more complex morphologies may show bigger benefits. For example, Sanskrit has a very large number of verb forms.

Spacy

Stemming and lemmatization are implementation dependent.

Spacy is a very modern & fast nlp library. Spacy is opinionated, in that it typically offers one highly optimized way to do something (whereas nltk offers a huge variety of ways, although they are usually not as optimized).

You will need to install it.

if you use conda:

```
conda install -c conda-forge spacy
```

if you use pip:

```
pip install -U spacy
```

You will then need to download the English model:

```
spacy -m download en_core_web_sm
```

In [9]:
```
import spacy
```

In [80]:
```
from spacy.lemmatizer import Lemmatizer
lemmatizer = Lemmatizer()
```

In [81]:
```
[lemmatizer.lookup(word) for word in word_list]
```

Out[81]:
```
['feet', 'foot', 'foots', 'footing']
```

Spacy doesn't offer a stemmer (since lemmatization is considered better-- this is an example of being opinionated!)

Stop words vary from library to library

In [12]:
```
nlp = spacy.load("en_core_web_sm")
```

In [13]:
```
sorted(list(nlp.Defaults.stop_words))[:20]
```

Out[13]:
```
['a',
 'about',
 'above',
 'across',
 'after',
 'afterwards',
 'again',
 'against',
 'all',
 'almost',
 'alone',
 'along',
 'already',
 'also',
 'although',
 'always',
 'am',
 'among',
 'amongst',
 'amount']
```

Exercise: What stop words appear in spacy but not in sklearn?

Exercise: What stop words appear in spacy but not in sklearn?

In [27]:
```
#Exercise:
```

Out[27]:
```
{'ca',
 'did',
 'does',
 'doing',
 'just',
 'make',
 'quite',
 'really',
 'regarding',
 'say',
 'unless',
 'used',
 'using',
 'various'}
```

Exercise: And what stop words are in sklearn but not spacy?

In [28]:
```
#Exercise:
```

Out[28]:
```
frozenset({'amoungst',
           'bill',
           'cant',
           'co',
           'con',
           'couldnt',
           'cry',
           'de',
           'describe',
           'detail',
           'eg',
           'etc',
           'fill',
           'find',
           'fire',
           'found',
           'hasnt',
           'ie',
           'inc',
           'interest',
           'ltd',
           'mill',
           'sincere',
           'system',
           'thick',
           'thin',
           'un'})
```

When to use these?

These were long considered standard techniques, but they can often **hurt** your performance **if using deep learning**. Stemming, lemmatization, and removing stop words all involve throwing away information.

However, they can still be useful when working with simpler models.

Another approach: sub-word units

SentencePiece library from Google

Data Processing

Next, scikit learn has a method that will extract all the word counts for us. In the next lesson, we'll learn how to write our own version of CountVectorizer, to see what's happening underneath the hood.

In [148…
```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
```

In [149…
```
import nltk
# nltk.download('punkt')
```

In [133…
```
# from nltk import word_tokenize

# class LemmaTokenizer(object):
#     def __init__(self):
#         self.wnl = stem.WordNetLemmatizer()
#     def __call__(self, doc):
#         return [self.wnl.lemmatize(t) for t in word_tokenize(doc)]
```

In [150…
```
vectorizer = CountVectorizer(stop_words='english') #, tokenizer=LemmaTokenize
```

In [151…
```
vectors = vectorizer.fit_transform(newsgroups_train.data).todense() # (docume
vectors.shape #, vectors.nnz / vectors.shape[0], row_means.shape
```

Out[151… (2034, 26576)

In [108…
```
print(len(newsgroups_train.data), vectors.shape)
```

```
2034 (2034, 26576)
```

In [109…
```
vocab = np.array(vectorizer.get_feature_names())
```

In [110…
```
vocab.shape
```

Out[110… (26576,)

In [111…
```
vocab[7000:7020]
```

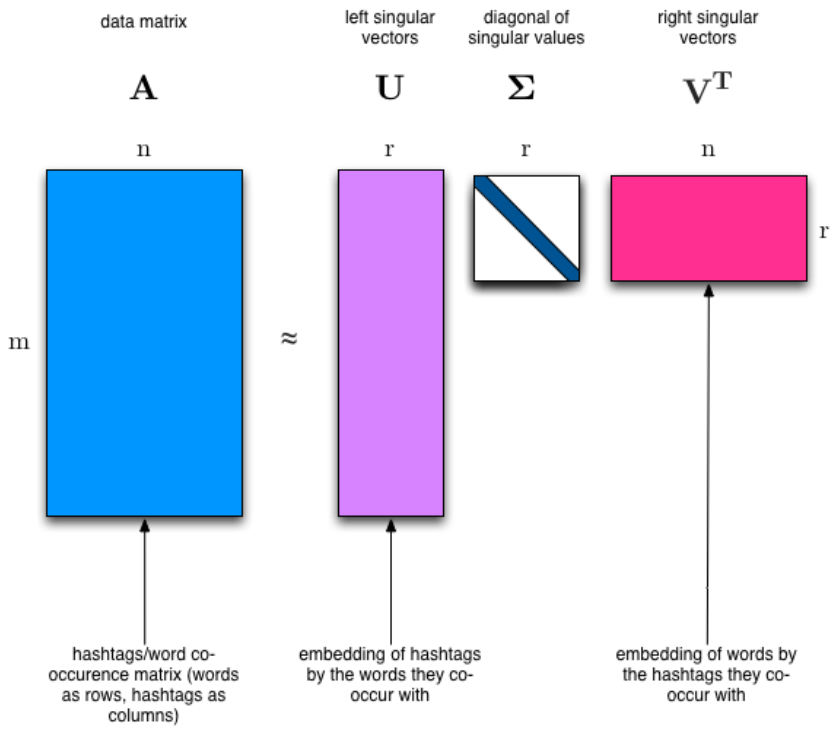
Out[111…
```
array(['cosmonauts', 'cosmos', 'cosponsored', 'cost', 'costa', 'costar',
       'costing', 'costly', 'costruction', 'costs', 'cosy', 'cote',
       'couched', 'couldn', 'council', 'councils', 'counsel',
       'counselees', 'counselor', 'count'], dtype='<U80')
```

Singular Value Decomposition (SVD)

"SVD is not nearly as famous as it should be." - Gilbert Strang

We would clearly expect that the words that appear most frequently in one topic would appear less frequently in the other - otherwise that word wouldn't make a good choice to separate out the two topics. Therefore, we expect the topics to be **orthogonal**.

The SVD algorithm factorizes a matrix into one matrix with **orthogonal columns** and one with **orthogonal rows** (along with a diagonal matrix, which contains the **relative importance** of each factor).



(source: [Facebook Research: Fast Randomized SVD](https://research.fb.com/fast-randomized-svd/))

SVD is an **exact decomposition**, since the matrices it creates are big enough to fully cover the original matrix. SVD is extremely widely used in linear algebra, and specifically in data science, including:

- semantic analysis
- collaborative filtering/recommendations (winning entry for Netflix Prize)
- calculate Moore-Penrose pseudoinverse
- data compression
- principal component analysis

Latent Semantic Analysis (LSA) uses SVD. You will sometimes hear topic modelling referred to as LSA.

In [152…
```
%time U, s, Vh = linalg.svd(vectors, full_matrices=False)
```

```
CPU times: user 1min 19s, sys: 10 s, total: 1min 29s
Wall time: 3.63 s
```

In [153…
```
print(U.shape, s.shape, Vh.shape)
```

```
(2034, 2034) (2034,) (2034, 26576)
```

Confirm this is a decomposition of the input.

In [157…
```
s[:4]
```

Out[157… `array([433.92698542, 291.51012741, 240.71137677, 220.00048043])`

In [158…
```
np.diag(np.diag(s[:4]))
```

Out[158… `array([433.92698542, 291.51012741, 240.71137677, 220.00048043])`

Answer

In [132…
```
#Exercise: confrim that U, s, Vh is a decomposition of `vectors`
```

Out[132… `True`

Confirm that U, V are orthonormal

Answer

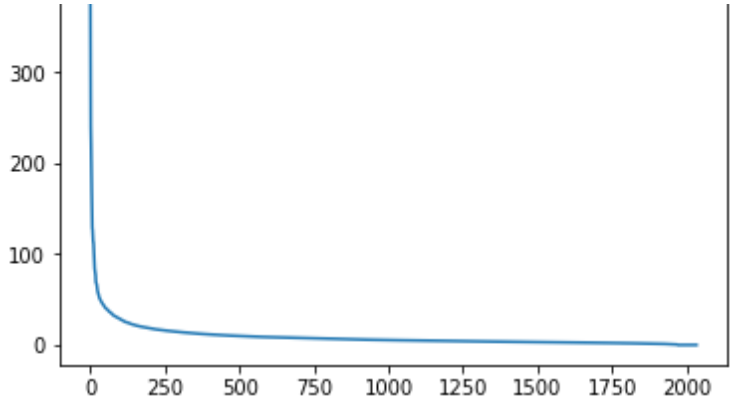
In [39]:
```
#Exercise: Confirm that U, Vh are orthonormal
```

Out[39]: `True`

Topics

What can we say about the singular values s?

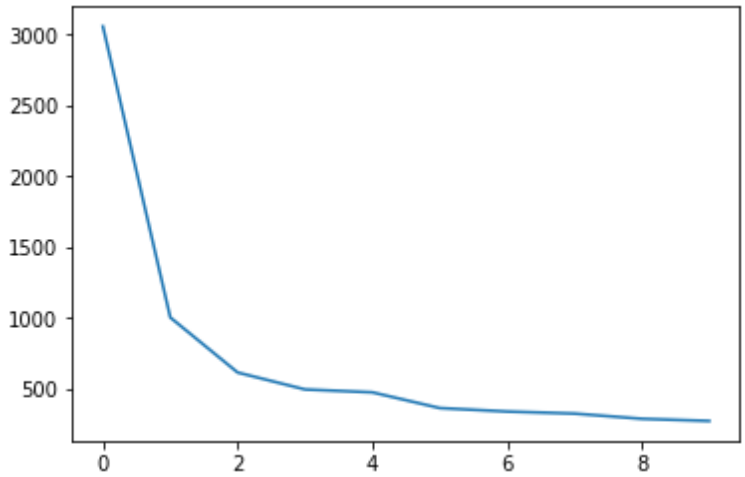
In [114…
```
plt.plot(s);
```



In [102…

```
plt.plot(s[:10])
```

Out[102… `[<matplotlib.lines.Line2D at 0x7f1781c7aa58>]`



In [104…

```
num_top_words=8

def show_topics(a):
    top_words = lambda t: [vocab[i] for i in np.argsort(t)[:-num_top_words-1
    topic_words = ([top_words(t) for t in a])
    return [' '.join(t) for t in topic_words]
```

In [115…

```
show_topics(Vh[:10])
```

Out[115…

```
['critus ditto propagandist surname galacticentric kindergarten surreal imagi
native',
 'jpeg gif file color quality image jfif format',
 'graphics edu pub mail 128 3d ray ftp',
 'jesus god matthew people atheists atheism does graphics',
 'image data processing analysis software available tools display',
 'god atheists atheism religious believe religion argument true',
 'space nasa lunar mars probe moon missions probes',
 'image probe surface lunar mars probes moon orbit',
 'argument fallacy conclusion example true ad argumentum premises',
 'space larson image theory universe physical nasa material']
```

We get topics that match the kinds of clusters we would expect! This is despite the fact that this is an **unsupervised algorithm** - which is to say, we never actually told the algorithm how our documents are grouped.

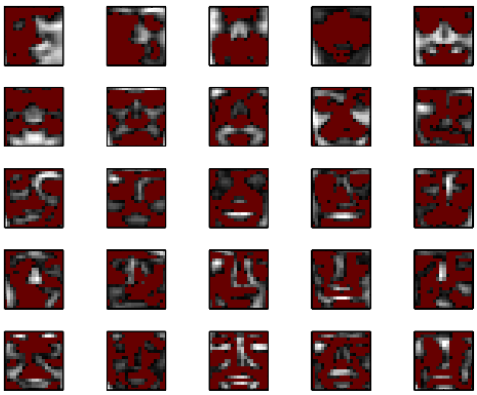
We will return to SVD in **much more detail** later. For now, the important takeaway is

that we have a tool that allows us to exactly factor a matrix into orthogonal columns and orthogonal rows.

Non-negative Matrix Factorization (NMF)

Motivation

Explaining face images by PCA[3]
Eigenfaces



*Red pixels indicate **negative values**! How to interpret this?*

[3]slide adapted from (Févotte, 2012).

Essid & Ozerov (TPT/Technicolor) A tutorial on NMF ICME 2014 10 / 170

(source: NMF Tutorial)

A more interpretable approach:

NMF outputs
Image example

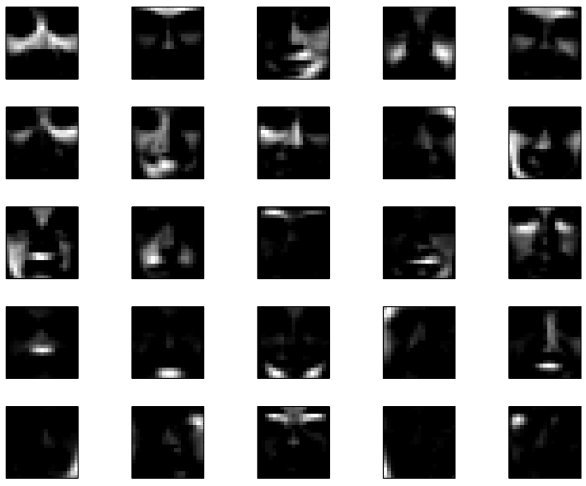


Illustration by C. Févotte

Essid & Ozerov (TPT/Technicolor) A tutorial on NMF ICME 2014

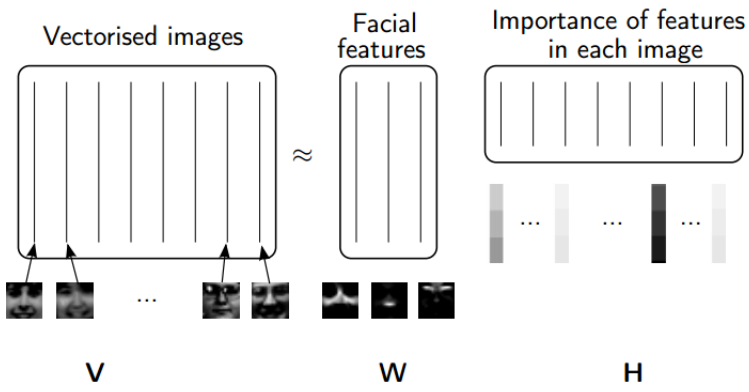
(source: NMF Tutorial)

Idea

Rather than constraining our factors to be *orthogonal*, another idea would to constrain them to be *non-negative*. NMF is a factorization of a non-negative data set V:

$$V = WH$$

into non-negative matrices W, H. Often positive factors will be **more easily interpretable** (and this is the reason behind NMF's popularity).

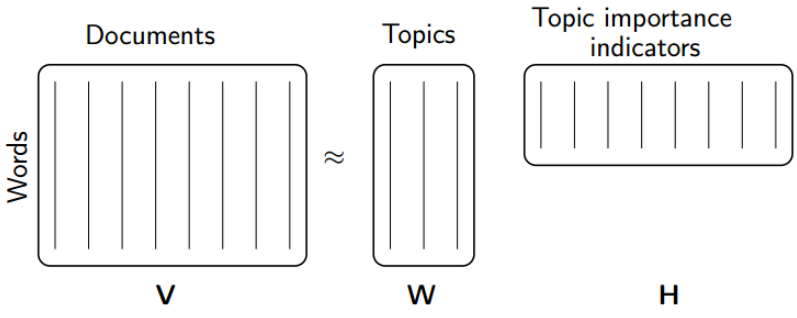


(source: NMF Tutorial)

Nonnegative matrix factorization (NMF) is a non-exact factorization that factors into one skinny positive matrix and one short positive matrix. NMF is NP-hard and non-unique. There are a number of variations on it, created by adding different constraints.

Applications of NMF

- Face Decompositions
- Collaborative Filtering, eg movie recommendations
- Audio source separation
- Chemistry
- Bioinformatics and Gene Expression
- Topic Modeling (our problem!)



(source: NMF Tutorial)

More Reading:

- The Why and How of Nonnegative Matrix Factorization

NMF from sklearn

We will use scikit-learn's implementation of NMF:

In [116…
```
m,n=vectors.shape
d=5  # num topics
```

In [117…
```
clf = decomposition.NMF(n_components=d, random_state=1)

W1 = clf.fit_transform(vectors)
H1 = clf.components_
```

In [118…
```
show_topics(H1)
```

Out[118…
```
['jpeg image gif file color images format quality',
 'edu graphics pub mail 128 ray ftp send',
 'space launch satellite nasa commercial satellites year market',
 'jesus god people matthew atheists does atheism said',
 'image data available software processing ftp edu analysis']
```

TF-IDF

Topic Frequency-Inverse Document Frequency (TF-IDF) is a way to normalize term counts by taking into account how often they appear in a document, how long the document is, and how commmon/rare the term is.

TF = (# occurrences of term t in document) / (# of words in documents)

IDF = log(# of documents / # documents with term t in it)

In [119…
```
vectorizer_tfidf = TfidfVectorizer(stop_words='english')
vectors_tfidf = vectorizer_tfidf.fit_transform(newsgroups_train.data) # (doc
```

In [160…
```
newsgroups_train.data[10:20]
```

Out[160…
```
["a\n\nWhat about positional uncertainties in S-L 1993e?   I assume we know w
here\nand what Galileo is doing within a few meters.   But without the\nHGA,
don't we have to have some pretty good ideas, of where to look\nbefore imagin
g?  If the HGA was working,  they could slew around\nin near real time (Less
speed of light delay).  But when they were\nimaging toutatis????  didn't some
one have to get lucky on a guess to\nfind the first images?   \n\nAlso, I ima
gine S-L 1993e will be mostly a visual image.  so how will\nthat affect the o
ther imaging missions.  with the LGA,  there is a real\ntight allocation of b
andwidth.   It may be premature to hope for answers,\nbut I thought i'd throw
it on the floor.",
 "I would like to program Tseng ET4000 to nonstandard 1024x768 mode by\nswitc
hing to standard 1024x768 mode using BIOS and than changing some\ntiming deta
ils (0x3D4 registers 0x00-0x1F) but I don't know how to\nselect 36 MHz pixel
clock I need. The BIOS function selects 40 MHz.\n\nIs there anybody who knows
where to obtain technical info about this.\nI am also interested in any other
technical information about Tseng ET4000\nand Trident 8900 and 9000 chipset
s.\n\n\t\t\tthanks very much",
 'In-Reply-To: <20APR199312262902@rigel.tamu.edu> lmp8913@rigel.tamu.edu (PRE
STON, LISA M)',
 "\n\n\n\nI'm not sure, but it almost sounds like they can't figure out where
the \n_nucleus_ is within the coma. If they're off by a couple hundred\nmile
s, well, you can imagine the rest...\n",
 "Hello,\n     I am looking to add voice input capability to a user interface
```

```
I am\ndeveloping on an HP730 (UNIX) workstation.  I would greatly appreciate
\ninformation anyone would care to offer about voice input systems that are
\neasily accessible from the UNIX environment. \n\n     The names or adresses
of applicable vendors, as well as any \nexperiences you have had with specifi
c systems, would be very helpful.\n\n     Please respond via email; I will po
st a summary if there is \nsufficient interest.\n\n\nThanks,\nKen\n\n\nP.S.
I have found several impressive systems for IBM PC's, but I would \nlike to a
void the hassle of purchasing and maintaining a separate PC if \nat all possi
ble.\n\n----------------------------------------------------------------------
----------\nKen Hinckley (kph2q@virginia.edu)\nUniversity of Virginia \nNeuro
surgical Visualization Laboratory",
 '\nIt was a test of the first reusable tool.\n\n\nPointy so they can find th
em or so they will stick into their pants better, and\nbe closer to their bra
ins?',
 '\nSize of armies, duration, numbers of casualties both absolute and as a\np
ercentage of those involved, geographical area and numbers of countries\ntoo,
are all measures of size.  In this case I\'d say the relevant\nstatistic woul
d be the number of combatants (total troops) compared to\ntotal casualties fr
om among the total civilian population in the\naffected geographical area.\n
\n\nVietnam and Korea might make good comparisons.\n\n\nWestern news in gener
al, but in particular the American "mass media":\nCBS, NBC, ABC, etc.  The ge
neral tone of the news during the whole\nwar was one of "those poor, poor Ira
qis" along with "look how precisely\nthis cruise missile blew this building t
o bits".\n\n\nI agree.\n\n\nPerhaps so.  And maybe the atomic bomb was a mist
ake too.  But that\'s easy\nto say from our "enlightened" viewpoint here in t
he 90\'s, right?  Back\nthen, it was *all-out* war, and Germany and Japan had
to be squashed.\nAfter all, a million or more British had already died, hundr
eds of \nthousands of French, a couple hundread thousand or so Americans, and
\nmillions of Russians, not to mention a few million Jews, Poles, and \nother
people of slavic descent in German concentration camps.  All \nthings conside
red, the fire-bombings and the atomic bomb were\nessential (and therefore jus
tified) in bringing the war to a quick\nend to avoid even greater allied loss
es.\n\nI, for one, don\'t regret it.\n\n\nSure.  And it\'s the people who suf
fer because of them.  All the more\nreason to depose these "entrenched politi
cal rulers operating in their\nown selfish interests"!  Or do you mean that t
his applies to the allies\nas well??\n\n\nI make no claim or effort to justif
y the misguided foreign policy of the\nWest before the war.  It is evident th
at the West, especially America,\nmisjudged Hussein drastically.  But once Hu
ssein invaded Kuwait and \nthreatened to militarily corner a significant port
ion of the world\'s\noil supply, he had to be stopped.  Sure the war could ha
ve been\nprevented by judicious and concerted effort on the part of the West
\nbefore Hussein invaded Kuwait, but it is still *Hussein* who is\nresponsibl
e for his decision to invade.  And once he did so, a\nstrong response from th
e West was required.\n\n\nWell, it\'s not very "loving" to allow a Hussein or
a Hitler to gobble up\nnearby countries and keep them.  Or to allow them to c
ontinue with mass\nslaughter of certain peoples under their dominion.  So, I
\'d have to\nsay yes, stopping Hussein was the most "loving" thing to do for
the\nmost people involved once he set his mind on military conquest.\n\nI men
tioned it.\n\nIf we hadn\'t intervened, allowing Hussein to keep Kuwait, then
it would\nhave been appeasement.  It is precisely the lessons the world learn
ed\nin WW2 that motivated the Western alliance to war.  Letting Hitler take\n
Austria and Czechoslavkia did not stop WW2 from happening, and letting\nHusse
in keep Kuwait would not have stopped an eventual Gulf War to\nprotect Saudi
Arabia.\n\n\nSure.  What was truly unfortunate was that they followed Hitler
in\nhis grandiose quest for a "Thousand Year Reich".  The consequences\nstemm
ed from that.\n\nWhat should I say about them?  Anything in particular?\n\n\n
\nSo?  It was the *policemen* on trial not Rodney King!!  And under American
\nlaw they deserved a jury of *their* peers!  If there had been black\noffice
rs involved, I\'m sure their would have been black jurors too.\nThis point (o
f allegedly racial motivations) is really shallow.\n\n\nSo?  It\'s "hard to i
magine"?  So when has Argument from Incredulity\ngained acceptance from the r
evered author of "Constructing a Logical\nArgument"?  Can we expect another r
evision soon??  :)  (Just kidding.)\n\n\nI have to admit that I wonder this t
oo.  But *neither* the prosecution\nnor the defense is talking.  So one canno
t conclude either way due to\nthe silence of the principals.  \n\n\nOK.  It c
```

ertainly seemed to me that there was excessive force involved.\nAnd frankly, the original "not guilty" verdict baffled me too. But then\nI learned that the prosecution in the first case did not try to convict\non a charge of excessive force or simple assault which they probably\nwould have won, they tried to get a conviction on a charge of aggravated\nassault with intent to inflict serious bodily harm. A charge, which\nnews commentators said, was akin to attempted murder under California\nlaw. Based on what the prosecution was asking for, it\'s evident that \nthe first jury decided that the officers were "not guilty". Note, \nnot "not guilty" of doing wrong, but "not guilty" of aggravated assault \nwith the *intent* of inflicting serious bodily harm. The seeds of the \nprosecutions defeat were in their own overconfidence in obtaining a \nverdict such that they went for the most extreme charge they could.\n\nIf the facts as the news commentators presented them are true, then\nI feel the "not guilty" verdict was a reasonable one.\n\nThanks mathew, I like the quote. Pretty funny actually. (I\'m a \nMonty Python fan, you know. Kind of seems in that vein.)\n\nOf course, oversimplifying any moral argument can make it seem\ncontradictory. But then, you know that already. \n\nRegards,',
 "<stuff deleted>\n\nYou mean like: seconds, minutes, hours, days, months, years. . . :-)\n\nRemember, the Fahrenheit temperature scale is also a centigrade scale. Some\nrevisionists tell the history something like this: The coldest point in a\nparticular Russian winter was marked on the thermometer as was the body\ntemperature of a volunteer (turns out he was sick, but you can't win 'em all).\nThen the space in between the marks on the thermometer was then divided into\nhundredths.\n\t\t\t\t\t\t\t\t:-)\n\nFWIW,\n\nDoug Page\n",
 "\nIt wasn't especially prominent, as I recall. However, quite possibly it's\nno longer on display; NASM, like most museums, has much more stuff than it\ncan display at once, and does rotate the displays occasionally.",
 "DM> Fact or rumor....? Madalyn Murray O'Hare an atheist who eliminated the \nDM> use of the bible reading and prayer in public schools 15 years ago is now\nDM> going to appear before the FCC with a petition to stop the reading of the\nDM> Gospel on the airways of America. And she is also campaigning to remove\nDM> Christmas programs, songs, etc from the public schools. If it is true\nDM> then mail to Federal Communications Commission 1919 H Street Washington DC\nDM> 20054 expressing your opposition to her request. Reference Petition number\n\nDM> 2493.\n\nFalse. This story has been going around for years. There's not a drop of\ntruth. Note that I don't care for O'Hare (O'Hair?) myself, but this\nis one thing she's not guilty of.\n"]

In [120…

```
W1 = clf.fit_transform(vectors_tfidf)
H1 = clf.components_
```

In [121…

```
show_topics(H1)
```

Out[121…

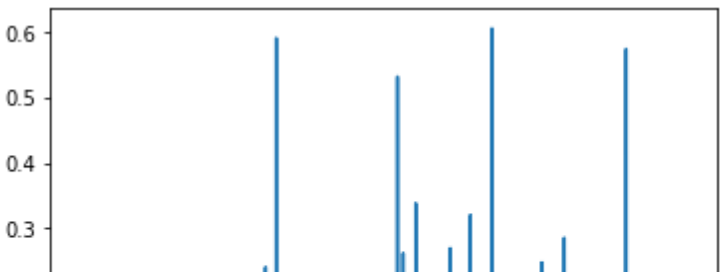
```
['people don think just like objective say morality',
 'graphics thanks files image file program windows know',
 'space nasa launch shuttle orbit moon lunar earth',
 'ico bobbe tek beauchaine bronx manhattan sank queens',
 'god jesus bible believe christian atheism does belief']
```

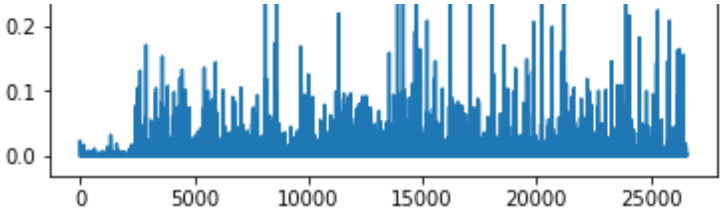
In [122…

```
plt.plot(clf.components_[0])
```

Out[122…

```
[<matplotlib.lines.Line2D at 0x7f15cb8ae0f0>]
```





In [99]:
```
clf.reconstruction_err_
```

Out[99]: 43.71292605795277

NMF in summary

Benefits: Fast and easy to use!

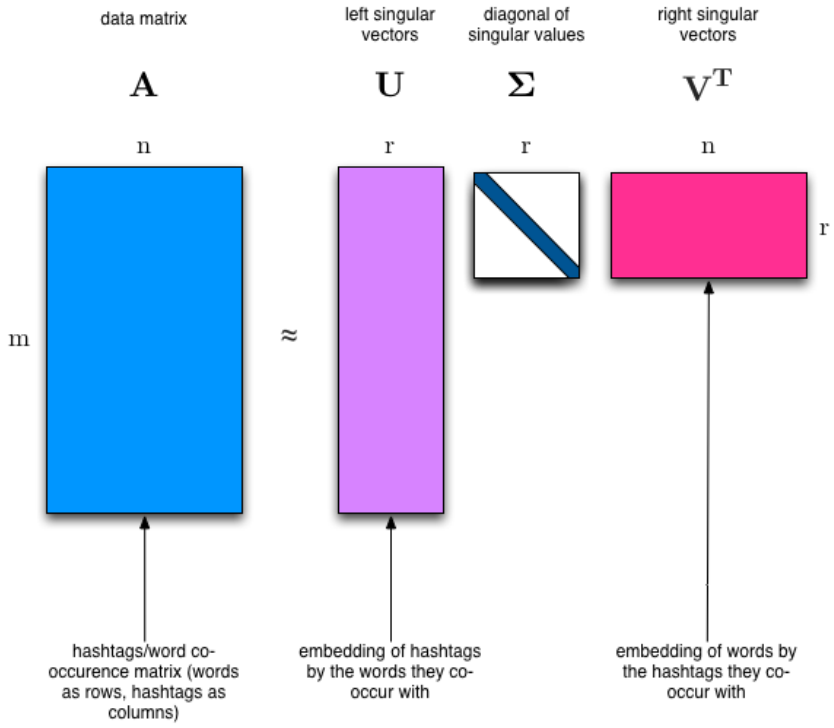
Downsides: took years of research and expertise to create

Notes:

- For NMF, matrix needs to be at least as tall as it is wide, or we get an error with fit_transform
- Can use df_min in CountVectorizer to only look at words that were in at least k of the split texts

Truncated SVD

We saved a lot of time when we calculated NMF by only calculating the subset of columns we were interested in. Is there a way to get this benefit with SVD? Yes there is! It's called truncated SVD. We are just interested in the vectors corresponding to the **largest** singular values.



(source: Facebook Research: Fast Randomized SVD)

Shortcomings of classical algorithms for decomposition:

- Matrices are "stupendously big"
- Data are often **missing or inaccurate**. Why spend extra computational resources when imprecision of input limits precision of the output?
- **Data transfer** now plays a major role in time of algorithms. Techniques the require fewer passes over the data may be substantially faster, even if they require more flops (flops = floating point operations).
- Important to take advantage of **GPUs**.

(source: Halko)

Advantages of randomized algorithms:

- inherently stable
- performance guarantees do not depend on subtle spectral properties
- needed matrix-vector products can be done in parallel

(source: Halko)

Timing comparison

In [123…
```
%time u, s, v = np.linalg.svd(vectors, full_matrices=False)
```

```
CPU times: user 1min 28s, sys: 9.3 s, total: 1min 37s
Wall time: 4.06 s
```

In [124…
```
from sklearn import decomposition
import fbpca
```

In [125…
```
%time u, s, v = decomposition.randomized_svd(vectors, 10)
```

```
CPU times: user 1min 18s, sys: 1.4 s, total: 1min 19s
Wall time: 3.03 s
```

Randomized SVD from Facebook's library fbpca:

In [126…
```
%time u, s, v = fbpca.pca(vectors, 10)
```

```
CPU times: user 26 s, sys: 644 ms, total: 26.7 s
Wall time: 1.19 s
```

For more on randomized SVD, check out my PyBay 2017 talk.

For significantly more on randomized SVD, check out the Computational Linear Algebra course.

End